

为什么 Agent 开发要学后端技术栈

2026-07-16

阅读需要 9 分钟

一、作为一个实习生，为什么想聊这个

3 月找暑期实习那会儿，agent 方向的面试主要问设计范式、MCP、Skill、RAG 这些。这些概念聊起来不难，面试也算混过去了。当时以为搞懂这些就是 agent 开发的全部——会设计 agent 架构、会用 MCP 接工具、懂 RAG 怎么搭，齐活。

6 月入职鹅厂，第一周 leader 甩过来一个需求：给线上的 agent 加重试和熔断。重试懂，熔断也听过，但翻了两天代码才意识到，这个所谓的"agent"，本质上就是一坨后端服务，LLM 只是其中最显眼的组件。RPC 调用、状态管理、并发控制、日志追踪、降级兜底——后端那套东西，一样不少地全在这里出现了，只是套了层"agent"的壳。

那两天特别受触动。在那之前，以为 agent 开发的全部就是搞懂那些概念和范式。但实际上这些东西可能只占一个生产级 agent 的 20%。剩下 80% 是网络、是并发、是存储、是可靠性、是可观测性——全是后端的活。

所以这篇文章不是教程。就是想把这几个月想通的一件事讲清楚：做 agent 开发，后端技术栈不是加分项，是地基。不补，短期 demo 能跑，长期一定撞墙。见过太多 demo 在线上跑两天就崩，崩的原因没一个跟概念有关，全是后端那套没做：超时没设、重试没做、状态没存、日志没打、成本没控。模型再聪明，工程不行，上线就是等着出事。

二、先把 Agent 拆开来：它真不是个聊天机器人

很多人第一次接触 agent，是从"让 LLM 帮忙查天气、订机票"那种 demo 开始的。看完觉得 agent 就是 LLM 加一堆工具函数，LLM 决定调哪个，调完拿结果继续推理。简单。

这个理解只对在 demo 层面。生产环境的 agent 拆开来：LLM 是大脑，负责理解、推理、决策；工具调用本质上是一次 RPC；记忆分短期（上下文）和长期（向量库）；编排决定 agent 每轮干啥、什么时候停，复杂点就是个工作流引擎；再加上存储和可观测层。把它和传统后端服务对比，几乎是 1:1 对应：工具调用对应 RPC 接口，短期记忆对应内存，长期记忆对应数据库加检索引擎，多 agent 协作对应分布式系统。没有一个组件是"agent 独有"的，全都是后端的老朋友。

其实看 agent 这个领域的关键词演变挺有意思。最早大家喊 prompt engineering，重心在写好 prompt；后来发现光 prompt 不够，上下文管不好模型再强也白搭，于是讲 context engineering；再后来大家意识到 LLM 外面那层工程骨架——工具调度、错误处理、重试熔断、状态恢复——才是能不能上线的关键，于是有了 harness engineering；到现在 agent 要自主跑多步，大家又开始讲 loop engineering。

前面的词一直在变，但 engineering 这个词从来没丢，而且比重越来越重——从"写好一段话"到"管好一坨上下文"到"给 LLM 套上工程骨架"到"设计一个会自己跑的循环"，工程含量一路上升。很多人只盯着前面那个词，却忽略了不管哪个阶段，真正决定能不能落地的都是后面那个 engineering。

demo 和生产差就差在这。demo 里写个 get_weather，LLM 调它，拿结果，返回用户，完事。生产里这个接口一上线，天气 API 挂了怎么办？超时设多少？要不要重试？重试会不会触发限流？API key 怎么管？这些问题 demo 一个不会遇到，生产一个都躲不掉。写 demo 是写 happy path，做生产是处理 unhappy path。而 unhappy path 的处理，全是后端工程那套东西。

三、网络这一关：工具调用不是函数调用

很多人觉得 agent 的工具调用就是"函数调用"。教程里写个 Python 函数，LLM 就能调，看起来跟本地函数没区别。

但本地函数是进程内的，几乎不会失败。agent 的工具调用绝大多数时候是一次网络请求，只要走网

络，后端那些老问题就全来了：认证鉴权（API key 不能写死、不能打日志）、超时（工具级、任务级、会话级每层都得有）、重试（要区分错误类型、指数退避、配合幂等）、限流（别把对方打挂或把自己 ban 掉）。

这些问题单拎出来都是后端工程师的日常。但 agent 有个特别的难点：工具调用的发起者是 LLM，不是人。LLM 行为不确定，可能这一轮调一次下一轮调五次，可能连续调同一个工具十次，可能一个会话把 API 额度跑光。没法像传统后端那样预估流量，只能靠工程兜底。

讲个真实例子。有个 agent，它的工具是“下单”。这个接口没做幂等，设计时假设用户点一下调一次。结果 agent 某次调用超时，LLM 没收到响应，自动重试，又重试。三次下单，三次都成功。用户买了一件商品，扣了三次钱。这个 bug 在 demo 阶段永远不会出现，上线后重试加非幂等，数据就脏了。最后是后端连夜加幂等键、对历史数据做对账才收拾干净。

agent 的工具调用本质是分布式 RPC，得按 RPC 的标准工程化。认证、超时、重试、限流、幂等，一个都不能少。这些不是高级技巧，是底线。

四、并发、状态、记忆：三个隐形大坑

并发、状态、记忆这三个坑，在 agent 里其实是同一个问题的三个面——怎么在不确定的执行流里管好“正在发生的事”和“已经发生的事”。

并发这块，最典型的是流式输出。LLM 是流式吐 token 的，agent 得一边收一边往前端推，同时还得在后台准备下一步工具调用。要是写阻塞代码——比如收 token 时同步查个数据库——整个 agent 就卡住了。实习第二个月改过一个 bug，现象是“用户多问几句就卡死”，原因就是某个工具调用是同步阻塞的，连接池打满，整个服务僵了。改成异步立刻好了。这种 bug 在 demo 阶段测不出来，因为 demo 就一个人用。

状态这块，短期状态最核心的是上下文窗口。很多人把它当“能塞多少字”的问题，其实它更像内存管理问题——窗口有限，塞满了要么截断要么压缩，哪些消息留、哪些压缩成摘要、哪些丢掉，是个策略问题，不是 LLM 能自己解决的。context engineering 这个词被提出来，很大程度上就是解决这个。长期状态就是大家说的“记忆”，主流是向量库加 RAG，但背后是个完整的检索系统：embedding 怎么选、chunk 怎么切、索引怎么建、rerank 怎么做——全是后端加信息检索的活。

多 agent 协作就更直接了，一旦进入多 agent——一个负责规划、一个负责执行、一个负责 review——立刻进入分布式系统。死锁、活锁、最终一致性，这些学院派的词全是真实问题。之前看一个多 agent demo，三个 agent 互相发消息发到死循环，CPU 拉满，这就是活锁。loop engineering 解决怎么让这个 loop 该跑该停，harness engineering 解决怎么给这个 loop 外面套一层可靠的壳——工具调度、错误兜底、状态恢复——让它出了问题能接住。

说白了，并发、状态、记忆这三件事合在一起，就是 harness engineering 要干的核心活——给 LLM 套上一层工程骨架，让它不只是个会说话的模型，而是个能跑、能稳、能恢复的系统。

五、可靠性和可观测性：让 agent 别那么黑盒

agent 最让人崩溃的一点：它会挂，而且挂得莫名其妙。传统后端挂了原因通常确定——OOM、连接打满、依赖超时。agent 不一样，挂的原因来自三方面：LLM 本身（幻觉、拒答、死循环）、工具（500、超时、脏数据）、编排逻辑（设计不对就死循环或卡死）。

面对这些，能做的就是可靠性工程那套全搬过来：失败重试但要区分错误类型，连续失败到阈值就熔断，工具不可用就降级给兜底回复，每层都设超时，实在跑不下去就转人工接管。这些就是后端微服务那套成熟的东西，agent 这里再用一遍。

但 agent 还多一个麻烦：可观测性。agent 是个巨大的黑盒，一次请求可能调了 LLM 三次、五个工具、两次向量库。没追踪的话出了问题根本不知道哪步炸的。日志每步都打，trace id 跨服务串起来，调用量、成功率、延迟、token 消耗都得有面板。

还有件容易忽略的事：成本追踪。LLM 按 token 计费，一轮几分钱听着不多，乘以用户量和调用次数，一个月可能是吓人的数字。有个团队上线后没做成本监控，月底账单出来直接傻眼——单个用户平均消耗是预期的五倍。token 就是钱，不做监控就是烧钱。

最后是评估。agent 测试比普通服务难，输出不确定，不能简单断言“输入 A 必须输出 B”。得搞评估集、跑离线评估、做在线 A/B。agent 没有评估系统，线上出问题根本发现不了。

六、结尾：说几句话

第一：别被 Agent 开发唬住。prompt engineering 是入门，不是天花板。它能帮你把 demo 跑起来，但跑起来之后的所有事——稳定性、性能、成本、可维护性——都跟 prompt 没啥关系，全是工程。不是说 prompt 不重要，但如果把全部精力都花在“怎么让 LLM 更聪明”上，而忽略了“怎么让 agent 这个系统更稳”，做的就永远是个 demo。

第二：后端基础该补的得补。按优先级：网络基础（HTTP、RPC、认证、超时、重试、幂等），这是一切的基础；并发编程（异步、协程、线程池、事件循环），流式输出和多用户并发都离不开；存储和缓存（Redis、关系库、向量库），记忆系统和状态管理的基础；分布式系统（消息队列、一致性、容错、编排），多 agent 协作必备；可观测性（日志、追踪、metrics），上线之后靠它活。

第三：短期看模型，长期看工程。这是这几个月最深的感受。模型在变强，每代新模型出来能力上限都在提高。但能力上限的提高，不等于做出来的 agent 就稳。模型再强，工具调用没做幂等，照样出三次下单的 bug；并发没做好，照样卡死；可观测没做，出了问题照样两眼一抹黑。

agent 这个领域，关键词从 prompt engineering 变到 context engineering，再到 harness engineering，再到 loop engineering，前面那个词一直在换，但 engineering 一直没丢。这不是巧合——agent 从来就是个工程问题，只是不同阶段套了不同的皮。模型决定 agent 能做多酷的事，工程决定 agent 能不能持续做多酷的事。前者决定上限，后者决定下限。新手容易盯着上限看，但下限稳不住，上限再高也没用。

这篇文章也是写给自己看的。如果你也在入门 agent，希望这些踩过的坑能帮你少走点弯路。后端这套东西看着枯燥，但它是 agent 能不能落地的命门。补上它，做的 agent 才有可能从 demo 走到生产。